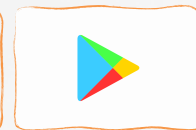




Kotlin

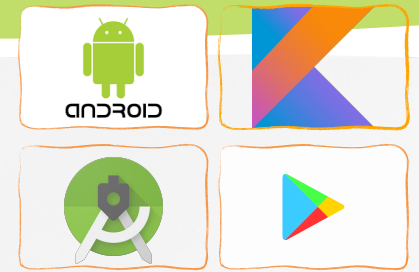
Dung Nguyen

nlhdung@icloud.com



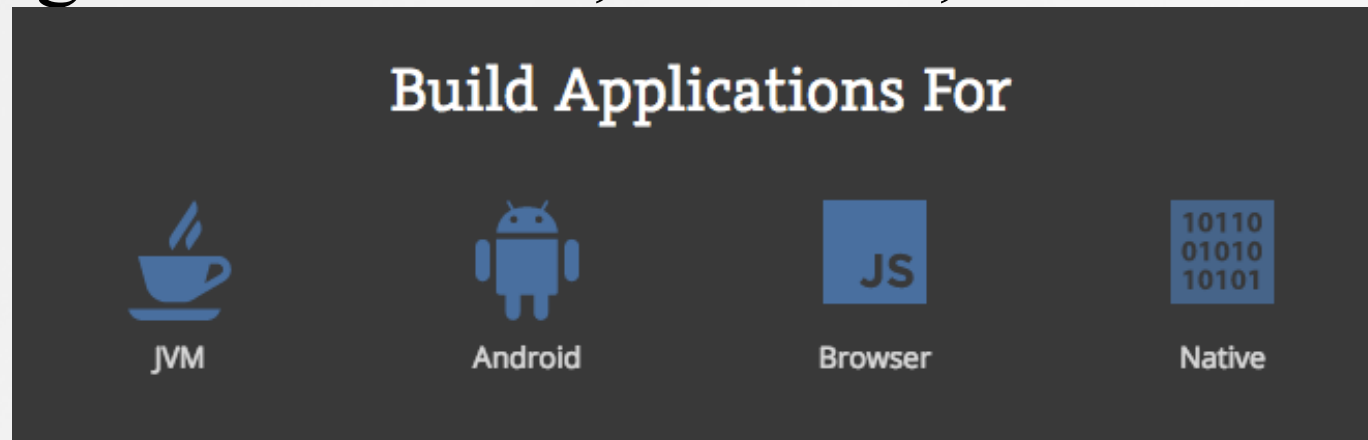
Content

- » What is it?
- » Why Kotlin?
- » Kotlin & Android Studio & IntelliJ IDEA
- » Kotlin's Syntax
- » Kotlin's Features

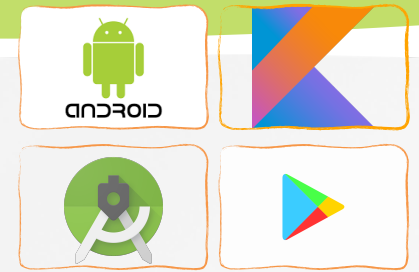


What is it?

- Kotlin is a statically typed programming language for the JVM, Android, Browser

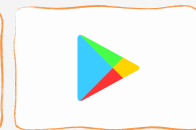


- An official programming language for Android
- It's interoperable with Java. We can use java code in Kotlin and Kotlin code in Java project.



Why Kotlin?

- **Concise:** Drastically reduce the amount of boilerplate code you need to write.
- **Safe:** Avoid entire classes of errors such as null pointer exceptions.
- **Interoperable:** Leverage existing frameworks and libraries of the JVM with 100% Java Interoperability.
- **Tool-friendly:** any Java IDE or build from command line.
- Great community and rapid development.

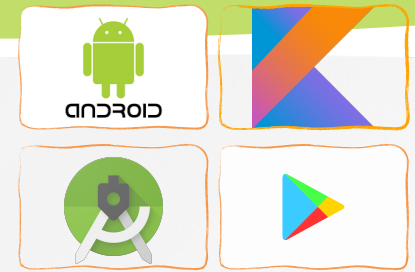


Why Kotlin?

//Java code

```
public class Aquarium {  
    private int mTemperature;  
  
    public Aquarium() { }  
  
    public int getTemperature() { return mTemperature; }  
    public void setTemperature(int mTemperature) {  
        this.mTemperature = mTemperature;  
    }  
  
    @Override  
    public String toString() {  
        return "Aquarium{" + "mTemperature=" + mTemperature + '}';  
    }  
}
```

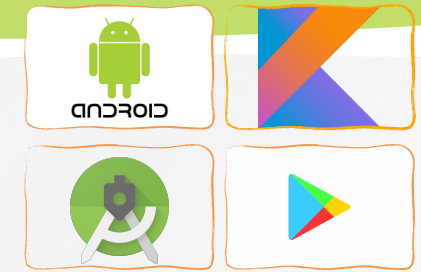
```
data class Aquarium (var temperature: Int = 0)
```



IntelliJ IDEA Setup

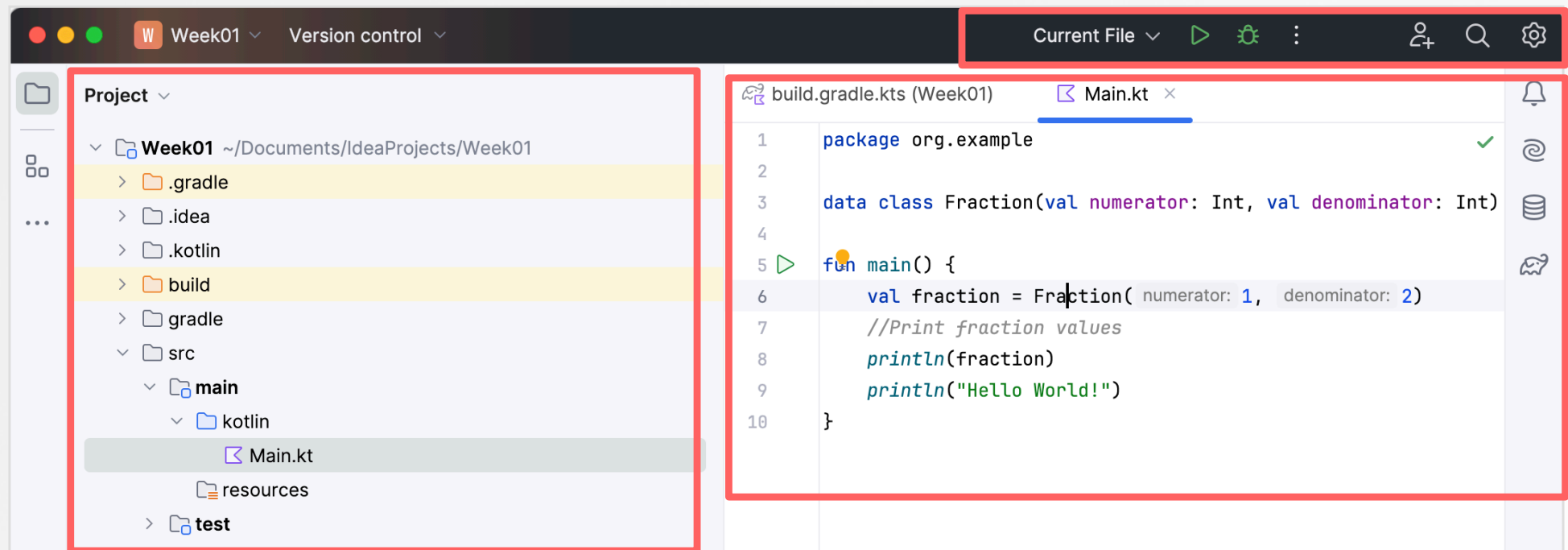
- Create a new Project
 - Input project name and select SDK version

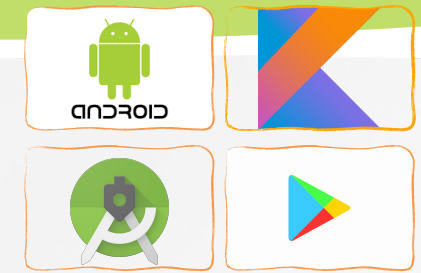
A screenshot of the IntelliJ IDEA 'New Project' dialog. The dialog is titled 'New Project' and has a search bar at the top left. Below the search bar, there are two sections: 'New Project' and 'Generators'. The 'New Project' section has four options: 'Java', 'Kotlin' (which is selected and highlighted in blue), 'Groovy', and 'Empty Project'. The 'Generators' section has five options: 'Maven Archetype', 'Jakarta EE', 'Spring Boot', 'JavaFX', and 'Quarkus'. On the right side of the dialog, there are several fields and checkboxes. The 'Name' field contains 'Week01'. The 'Location' field contains '~/Documents/IdeaProjects' and has a folder icon on the right. Below the 'Location' field, it says 'Project will be created in: ~/Documents/IdeaProjects/Week01'. There is a checkbox for 'Create Git repository' which is unchecked. The 'Build system' section has three buttons: 'IntelliJ', 'Maven', and 'Gradle'. The 'JDK' field is a dropdown menu showing '16 version 16.0.2'. The 'Gradle DSL' section has two buttons: 'Kotlin' and 'Groovy'. At the bottom, there is a checkbox for 'Add sample code' which is checked.



IntelliJ IDEA Setup (cont.)

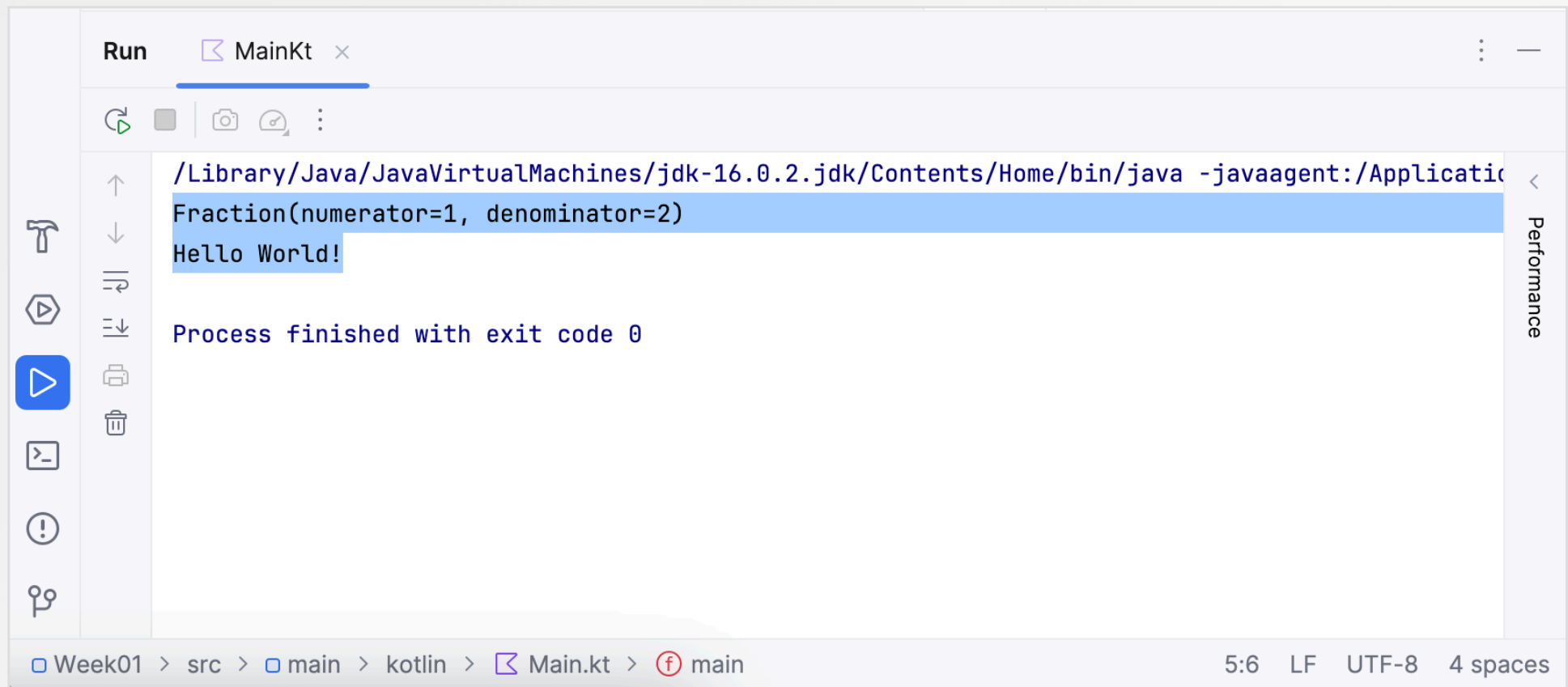
- The new project created with the following folder structure:

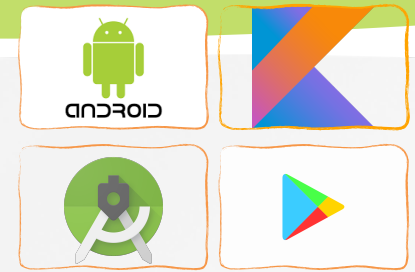




IntelliJ IDEA Setup (cont.)

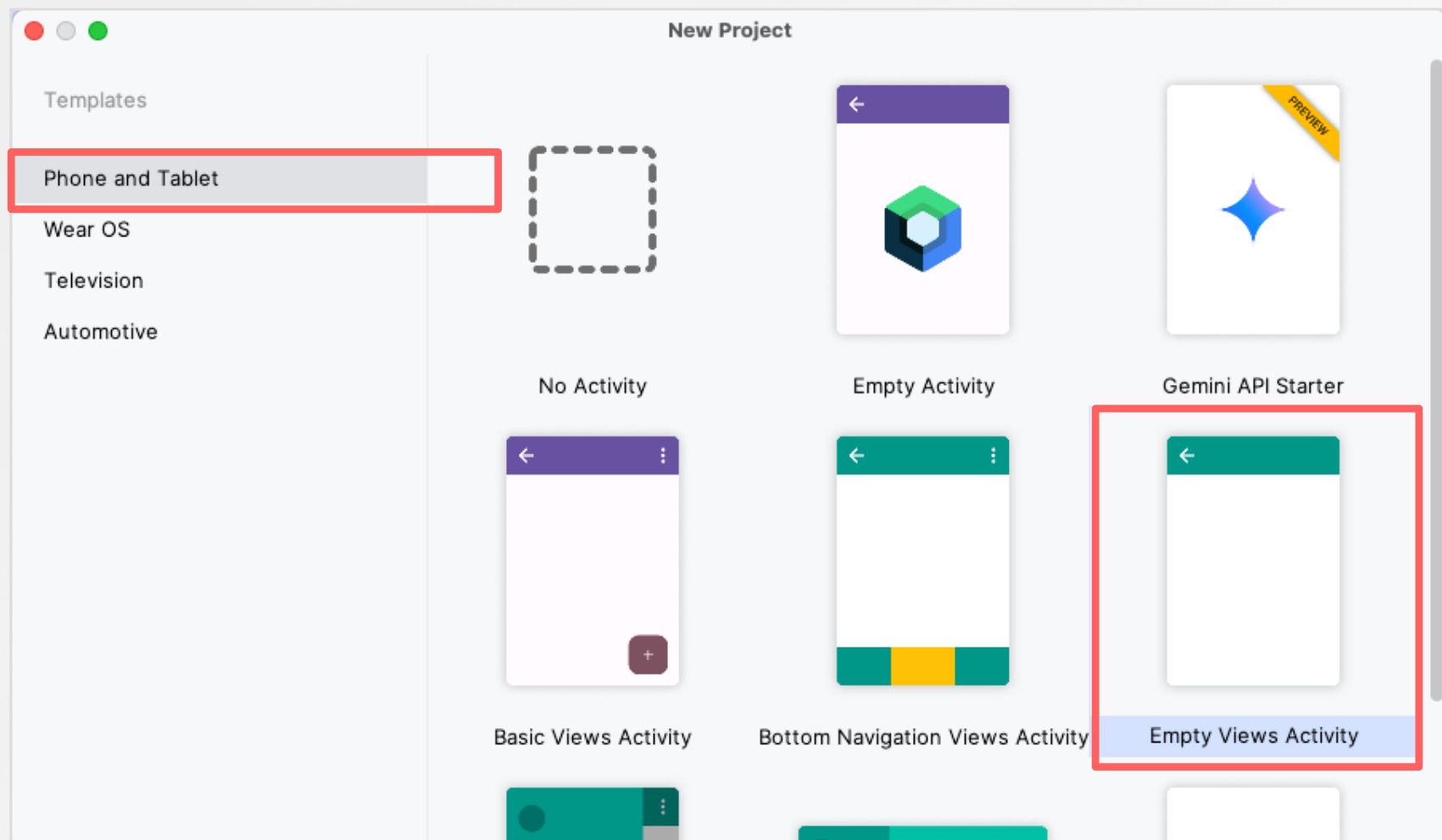
- The result in the **Run** tool window





Android Studio Setup

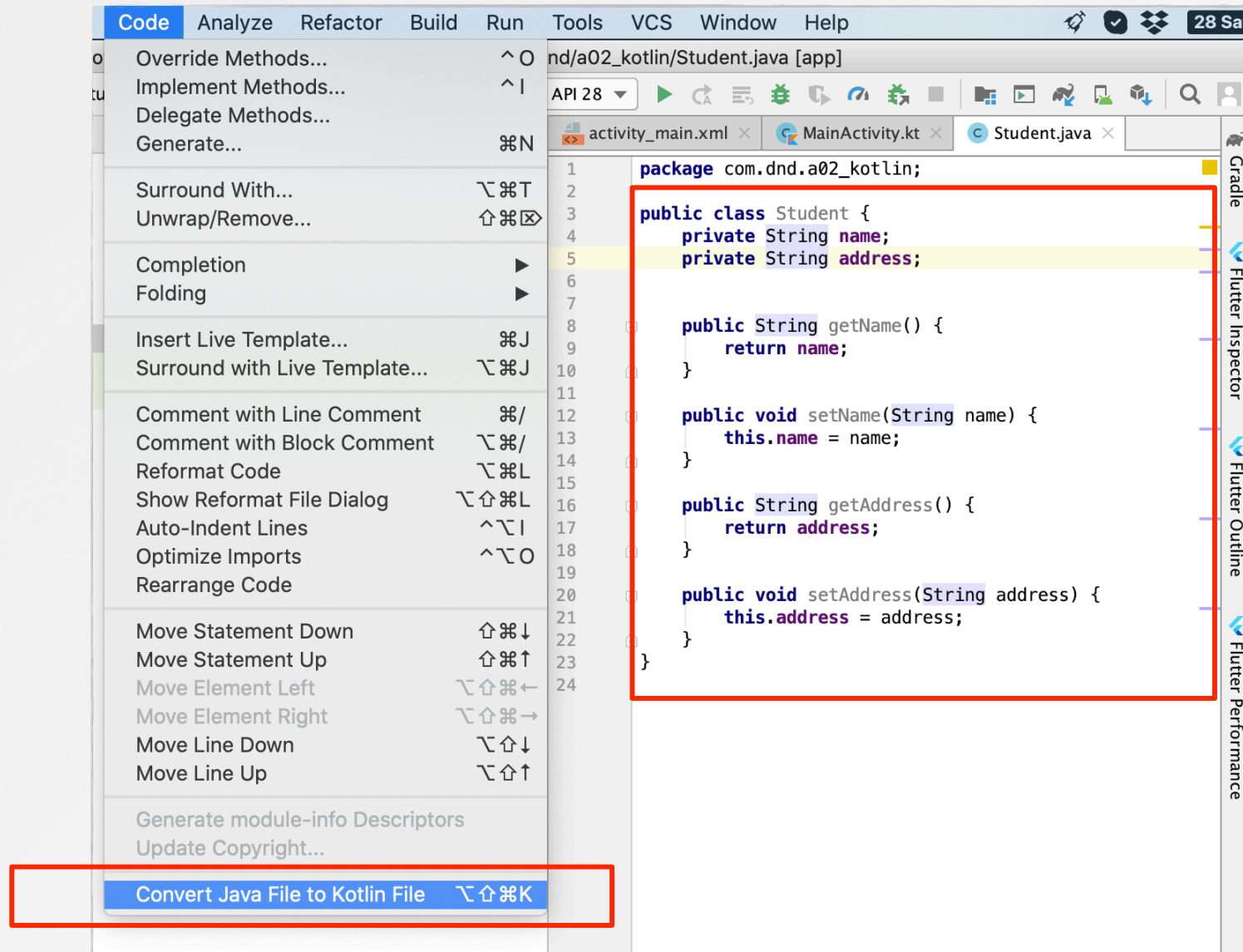
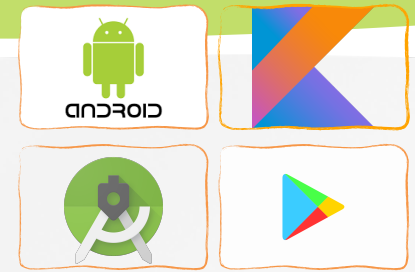
- Create new Android project



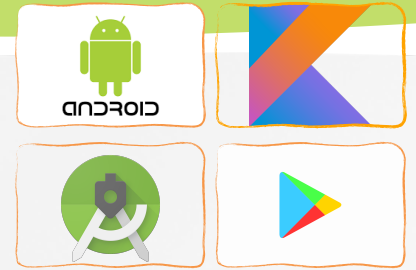
A 2x2 grid of icons. Top-left: The green Android robot logo above the word 'ANDROID' in black capital letters. Top-right: The Google Assistant logo, a colorful geometric shape made of triangles. Bottom-left: The Google Assistant logo, a green circle with a white robot head inside. Bottom-right: The Google Play Store logo, a colorful triangle made of smaller triangles.

10

Android Studio Setup (cont.)

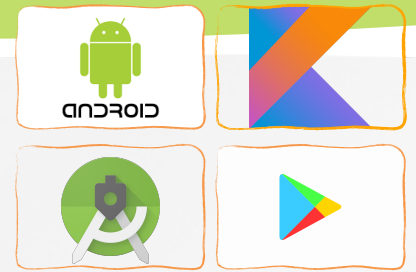


Android Studio Setup (cont.)



```
Student.kt x
1 package com.dnd.a02_kotlin
2
3 class Student {
4     var name: String? = null
5     var address: String? = null
6 }
```

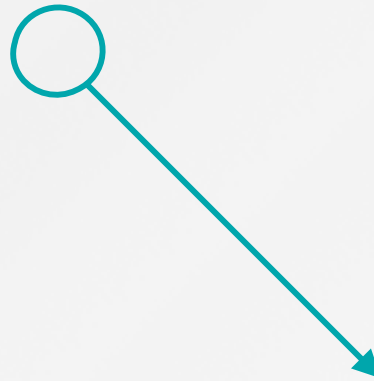
<https://kotlinlang.org/docs/tutorials/kotlin-android.html>



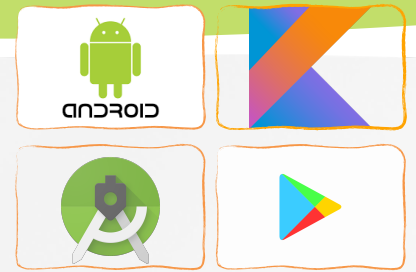
What does it look like?

Final

val x : Int = 1



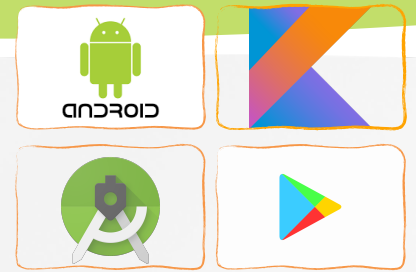
Semicolons are optional



What does it look like?

Type is
optional

```
val x : Int = 1
```

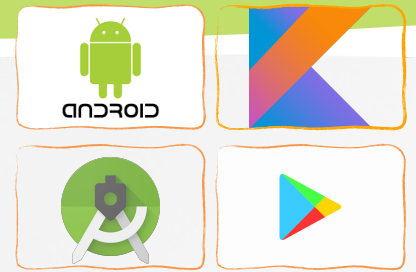


What does it look like?

Type is
optional

```
val x : Int = 1
```

```
val y = 1
```



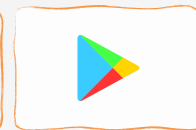
What does it look like?

```
val x : Int = 1
```

```
val y = 1
```

normal
variable

```
var z : Int
```

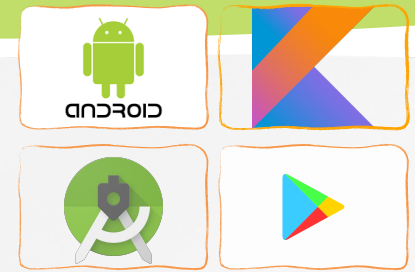


Compare conditions & booleans

```
val numberOfFish = 50
val numberOfPlants = 23
if (numberOfFish > numberOfPlants) {
    println("Good ratio!")
} else {
    println("Unhealthy ratio")
}
```

```
val fish = 50
if (fish in 1..100) {
    println(fish)
}
```

```
when (numberOfFish) {
    0 -> println("Empty tank")
    in 1..39 -> println("Got fish!")
    else -> println("That's a lot of fish!")
}
```



Lists and Arrays

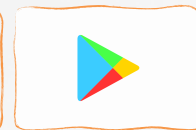
Instance of
integer ArrayList

No need “new”
attribute

```
var z = ArrayList<Int>()
```

```
val school = listOf("mackerel", "trout", "halibut")  
println(school)
```

```
val myList = mutableListOf("tuna", "salmon", "shark")  
myList.remove("shark")
```



Make loops

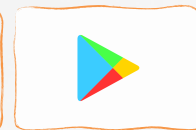
```
val school = arrayOf("shark", "salmon", "minnow")  
for (element in school) {  
    print(element + " ")  
}
```

```
for ((index, element) in school.withIndex()) {  
    println("Item at $index is $element\n")  
}
```

⇒ Item at 0 is shark

Item at 1 is salmon

Item at 2 is minnow



Make loops

```
for (i in 1..5) print(i)
```

⇒ 12345

```
for (i in 5 downTo 1) print(i)
```

⇒ 54321

```
for (i in 3..6 step 2) print(i)
```

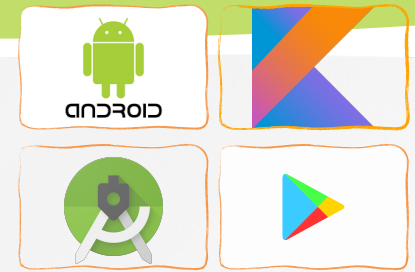
⇒ 35

```
for (i in 'd'..'g') print (i)
```

⇒ defg

Kotlin's Features





String template

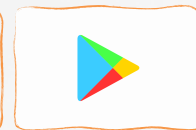
```
val packageName = "com.facebook.katana"
```

```
val language = "en"
```

```
val googlePlayUrl = "https://play.google.com/store/apps/  
details?id=$packageName&hl=$language"
```

```
print(googlePlayUrl)
```

```
//https://play.google.com/store/apps/details?  
id=com.facebook.katana&hl=en
```



Functions

fun
keyword

**Default
param(s)**

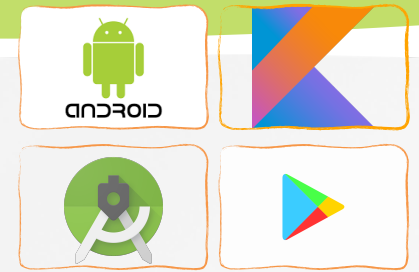
Variable
number of arguments
vararg

```
fun printSum(x: Int, y: Int = 0) {  
    print(x + y)  
}
```

return
type

```
fun getSum(x: Int, y: Int) : Int {  
    return x + y  
}
```

```
fun foo(vararg numbes: Int) {  
    for(item in numbes) {  
        println(item)  
    }  
}
```



Compact functions (Single-expression)

```
fun theAnswer(): Int {  
    return 42  
}
```

Need to
reduce

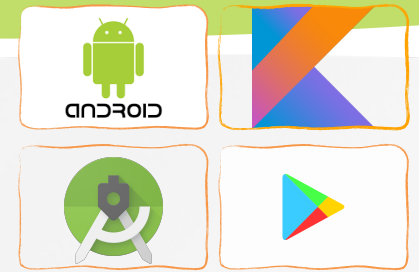
```
fun theAnswer() = 42
```

KOTLINIZED!

```
fun isTooHot(temperature: Int) = temperature > 30
```

```
fun isDirty(dirty: Int) = dirty > 30
```

```
fun isSunday(day: String) = day == "Sunday"
```

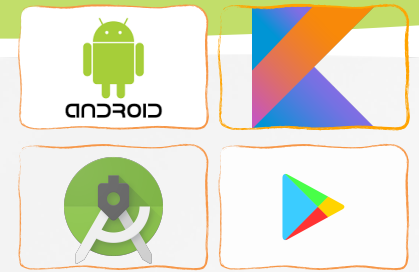


Lambdas

- A lambda is an expression that makes a function
 - Instead of declaring a named function, you declare a function that has no name
- => It can be passed as data

```
var dirtyLevel = 20  
val waterFilter = { dirty : Int -> dirty / 2 }  
println(waterFilter(dirtyLevel))
```

```
val waterFilter: (Int) -> Int = { dirty -> dirty / 2 }
```

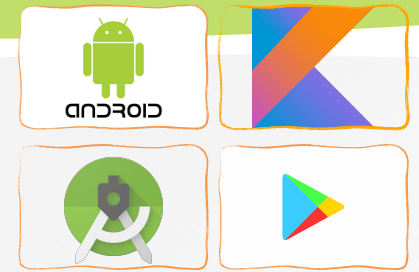


Lambdas (cont.)

- Higher-order function

```
fun updateDirty(dirty: Int, operation: (Int) -> Int): Int {  
    return operation(dirty)  
}
```

```
val waterFilter: (Int) -> Int = { dirty -> dirty / 2 }  
println(updateDirty(30, waterFilter))
```

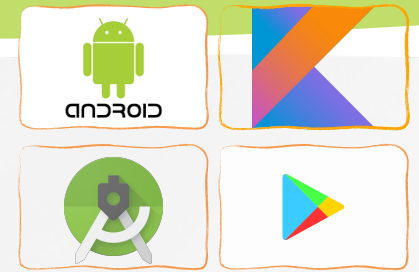


Lambdas (cont.)

```
button.setOnClickListener(object : View.OnClickListener {  
    override fun onClick(v: View?) {  
        //Button click  
    }  
})
```

Parameter
is optional

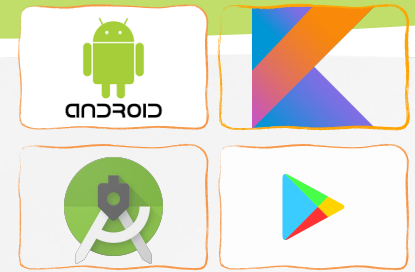
```
button.setOnClickListener { view ->  
    //Button click  
}
```



Lambdas (cont.)

```
button.setOnClickListener(object : View.OnClickListener {  
    override fun onClick(v: View?) {  
        //Button click  
    }  
})
```

```
button.setOnClickListener {  
    //Button click  
}
```

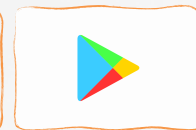
Classes and Objects

» Class Declaration

- » Using **var** or **val**, and Kotlin also creates the getters and setters automatically
- » Default is **val**

```
class ClassName(Param-Declaration1, Var-Name2:Var-Type2, ...) {  
    [Class-Body]  
}
```

```
class Student(var name:String, val grade:Int) {  
    fun changeValues(){  
        name = "new name"  
        grade = 2  
    }  
}
```



Classes and Objects (cont.)

» During an instance initialization, the *init* blocks are executed in the same order as they appear in the class body

```
class InitOrderDemo(name: String) {  
    val firstProperty = "First property: $name".also(::println)  
  
    init {  
        println("First initializer block that prints ${name}")  
    }  
  
    val secondProperty = "Second property: ${name.length}".also(::println)  
  
    init {  
        println("Second initializer block that prints ${name.length}")  
    }  
}
```



Classes and Objects (cont.)

Secondary constructors

» The secondary constructors, which are prefixed with **constructor**:

```
class Person {  
    constructor(address: String) {  
        println(address)  
    }  
}  
  
fun main() {  
    val obj = Person("227 Nguyễn Văn Cừ")  
}
```

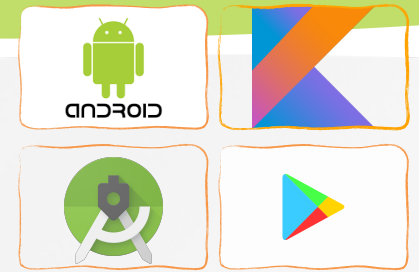


Classes and Objects (cont.)

Secondary constructors

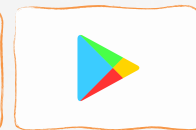
» Each secondary constructor needs to delegate to the primary constructor

```
class Person (name:String) {  
    constructor(name:String, address: String) :this(name) {  
        println(address)  
    }  
    init {  
        println(name)  
    }  
}  
  
fun main() {  
    val obj = Person("Kotlin", "227 Nguyễn Văn Cừ")  
}
```



Visibility modifiers

- » ***public (default)*** means visible outside the class. Everything is public by default, including variables and methods of the class.
- » ***internal*** means it will only be visible within that module. A **module** is a set of Kotlin files compiled together, for example, a library or application.
- » ***private*** means it will only be visible in that class (or source file if you are working with functions).
- » ***protected*** is the same as **private**, but it will also be visible to any subclasses.



Subclasses and inheritance

```
open class Aquarium (open var length: Int = 100, open var width: Int = 20,  
open var height: Int = 40) {  
    open var volume: Int  
        get() = width * height * length / 1000  
        set(value) {  
            height = (value * 1000) / (width * length)  
        }  
  
    open val shape = "rectangle"  
}
```

```
class TowerTank (override var height: Int, var diameter: Int):  
    Aquarium(height = height, width = diameter, length = diameter) {
```



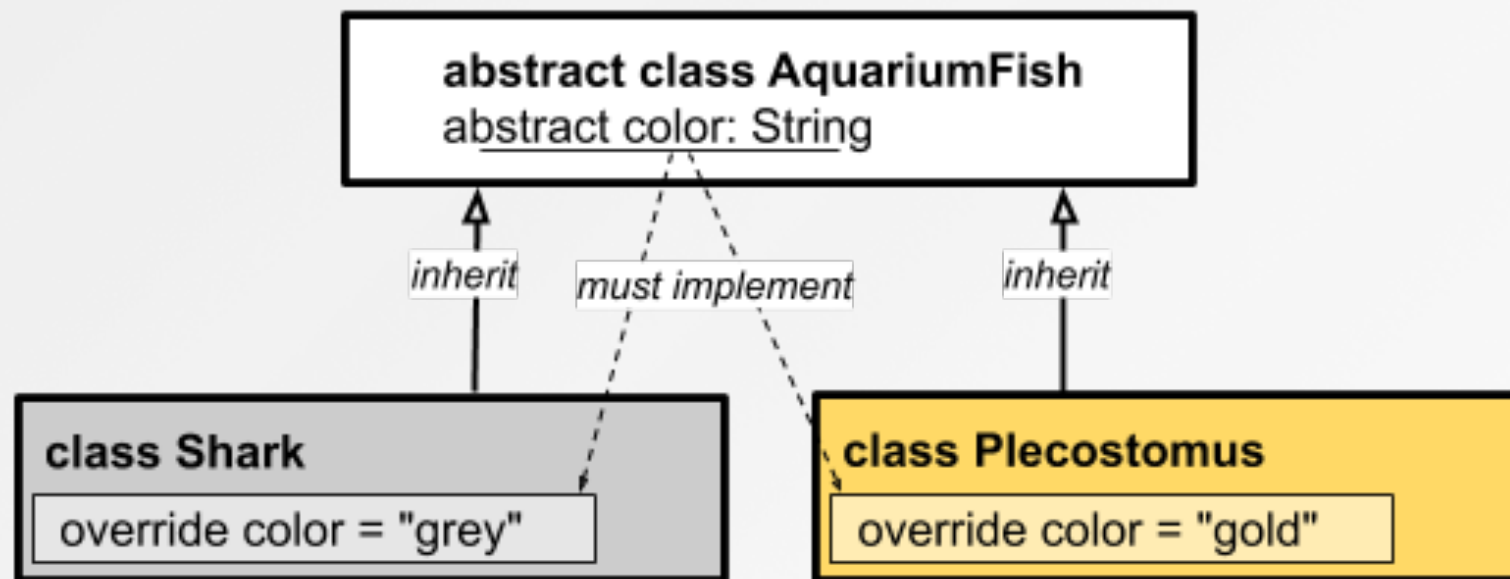
Subclasses and inheritance

```
class TowerTank (override var height: Int, var diameter: Int):  
    Aquarium(height = height, width = diameter, length = diameter) {  
        override var volume: Int  
            // ellipse area =  $\pi * r1 * r2$   
            get() = (width/2 * length/2 * height / 1000 * PI).toInt()  
            set(value) {  
                height = ((value * 1000 / PI) / (width/2 * length/2)).toInt()  
            }  
  
        override val shape = "cylinder"  
    }
```

Compare abstract classes and interfaces



One abstract class, two subclasses

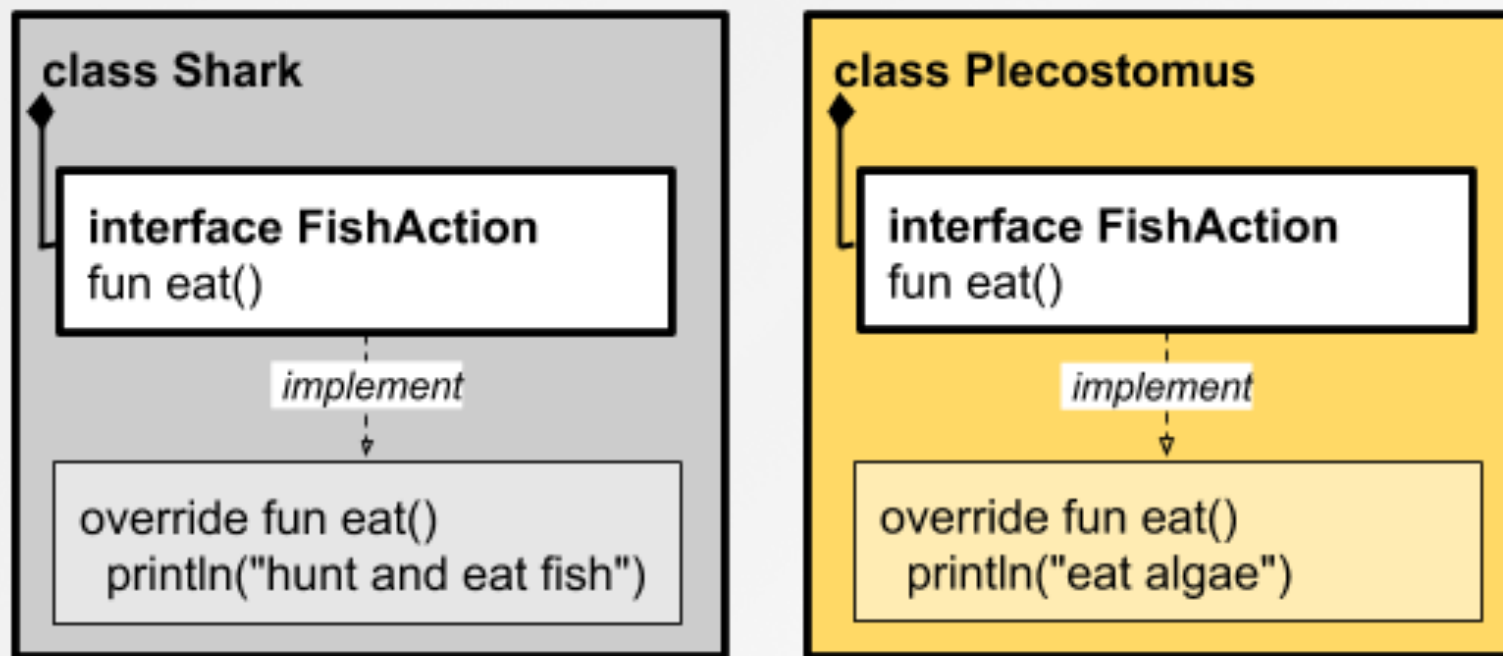


```
abstract class AquariumFish {
    abstract val color: String
}
class Shark: AquariumFish() { override val color = "gray" }
class Plecostomus: AquariumFish() { override val color = "gold" }
```




Compare abstract classes and interfaces

Two classes, one interface



```
interface FishAction {  
    fun eat()  
}
```

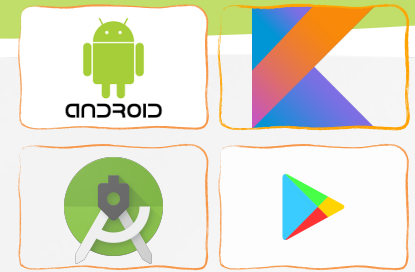
Compare abstract classes and interfaces



```
class Shark: AquariumFish(), FishAction {  
    override val color = "gray"  
    override fun eat() {  
        println("hunt and eat fish")  
    }  
}
```

```
class Plecostomus: AquariumFish(), FishAction {  
    override val color = "gold"  
    override fun eat() {  
        println("eat algae")  
    }  
}
```

```
interface AquariumAction  
{  
    fun eat()  
    fun jump()  
    fun clean()  
    fun catchFish()  
    fun swim() {  
        println("swim")  
    }  
}
```



Data Classes

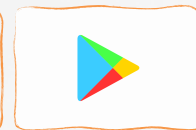
Just add
data

data class User(**val** name: String, **val** age: Int)

Getter	Setter
Equals	HashCode
toString	Copy

- they're similar to structs in some other languages
- they have utilities for **printing** and **copying**

<https://kotlinlang.org/docs/reference/data-classes.html#data-classes>



Data Classes (cont.)

```
data class User(val name: String, val age: Int)
```

```
val user = User("Kotlin", 26)
```

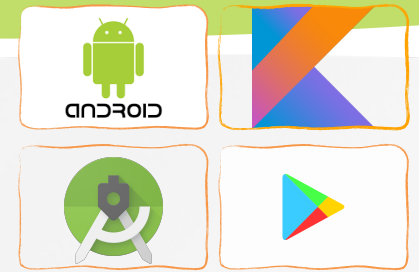
```
val otherUser = user.copy(age = 55)
```

```
println(user.toString())
```

```
println(otherUser.toString())
```

```
//User(name=Kotlin, age=26)
```

```
//User(name=Kotlin, age=55)
```



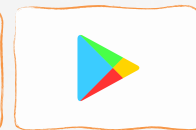
Data Classes (cont.)

```
data class Person(val name: String) {  
    var age: Int = 0  
}
```

```
val person1 = Person("John")  
val person2 = Person("John")  
person1.age = 10  
person2.age = 20
```

```
println(person1==person2)  
//true
```

The copy(), equals(), and other data class utilities only reference properties defined in the primary constructor.



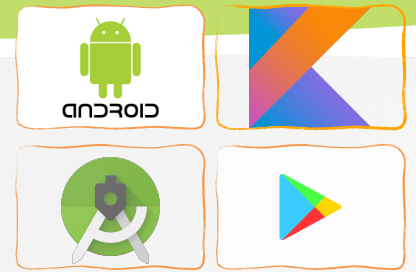
Enum Classes

```
enum class Direction {  
    NORTH, SOUTH, WEST, EAST  
}
```

```
fun main() {  
    val direction = Direction.SOUTH  
  
    println(direction.name) //SOUTH  
    println(direction.ordinal) //1  
}
```

```
enum class Direction(val degrees: Int)  
{  
    NORTH(0), SOUTH(180),  
    EAST(90), WEST(270)  
}
```

```
fun main() {  
    println(Direction.EAST.name)  
    println(Direction.EAST.ordinal)  
    println(Direction.EAST.degrees)  
}
```



Smart Cast

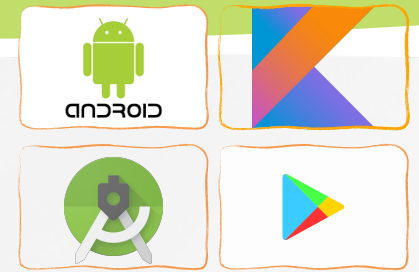
[//https://kotlinlang.org/docs/reference/typecasts.html#smart-casts](https://kotlinlang.org/docs/reference/typecasts.html#smart-casts)

```
fun main(){  
    demo("Hello World")  
}
```

```
fun demo(x: Any)  
{  
    if (x is String) {  
        print(x.length) // x is automatically cast to String  
    }  
}
```

is ; !is

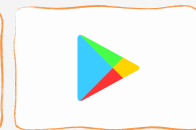
No need
to cast it!



Null safety

```
fun main() {  
    var str : String = "Hello Kotlin"  
    str = null // ?  
}
```

```
fun main() {  
    var str : String? = "Hello Kotlin"  
    str = null // ?  
}
```

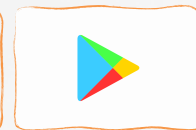
"Unsafe" and "Safe" cast operator

```
val y = 1
```

```
val x: String = y as String //not nullable
```

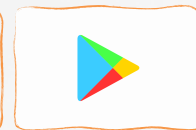
```
var x2: String? = y as String? //nullable
```

```
var x3: String? = y as? String? //safe cast with as?
```



Checking for **null** in conditions

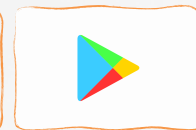
```
val b: String? = "Kotlin"  
if (b != null && b.length > 0) {  
    print("String of length ${b.length}")  
}else {  
    print("Empty string")  
}
```



Safe Calls

```
val a = "Kotlin"  
val b: String? = null  
println(b?.length)  
println(a.length)  
// 6  
// Int?
```

```
// Safe calls are useful in chains.  
// Such a chain returns null if any of the properties  
// in it is null  
bob?.department?.head?.name
```



Safe Calls (cont.)

// Using let

```
val listWithNulls: List<String?> = listOf("Kotlin", null)
for (item in listWithNulls) {
    item?.let { println(it) } // prints Kotlin and ignores null
}
```



Elvis Operator

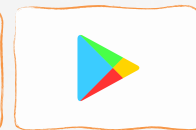
```
val b: String? = null
```

*// Using **if**-expression*

```
val l: Int = if (b != null) b.length else -1
```

*// Using **Elvis Operator***

```
val l2 = b?.length ?: -1
```



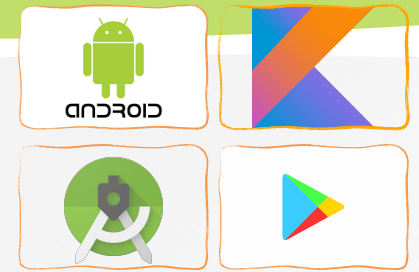
The !! Operator

// The operator (!!) converts any value to a non-null type

```
val b: String? = null
```

```
val l = b!!.length
```

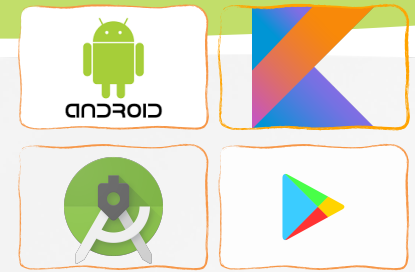
//Exception in thread “main” kotlin.KotlinNullPointerException



Lazy Property

```
data class Person(val name: String) {  
    var age: Int = 0  
  
    val address: String by lazy  
    {  
        "227 Nguyen Van Cu"  
    }  
}
```

Won't
be executed,
until first call



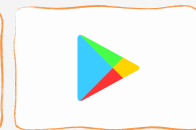
Extension Functions

```
import java.time.*
import java.time.format.DateTimeFormatter
import java.util.*

fun main(){
    var formatter = DateTimeFormatter.ofPattern("dd-MM-yyyy")
    var localDate = LocalDate.parse("31-12-2019", formatter)
    val date = localDate.toDate()

    println(localDate)    //2019-12-31
    println(date)         //Tue Dec 31 00:00:00 ICT 2019
}

fun LocalDate.toDate() : Date {
    return Date.from(this.atStartOfDay().atZone(ZoneId.systemDefault())
        .toInstant())
}
```

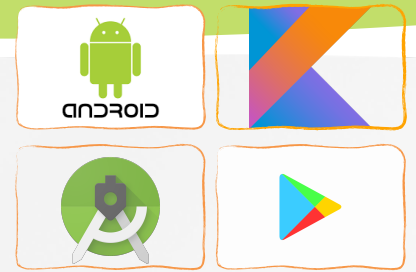
With

- Calling multiple methods on an object instance

```
class Turtle {  
    fun penDown(){  
        print("↓")  
    }  
    fun penUp(){  
        print("↑")  
    }  
    fun turnLeft(){  
        print("←")  
    }  
}
```

```
val yourTurtle = Turtle()  
yourTurtle.penDown()  
yourTurtle.turnLeft()  
yourTurtle.penUp()
```

```
val myTurtle = Turtle()  
with(myTurtle) {  
    penDown()  
    turnLeft()  
    penUp()  
}
```

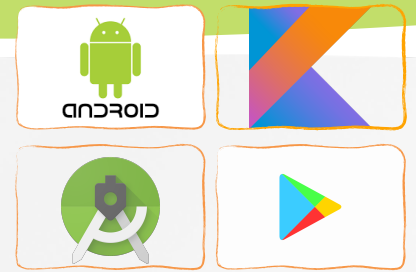


Exceptions

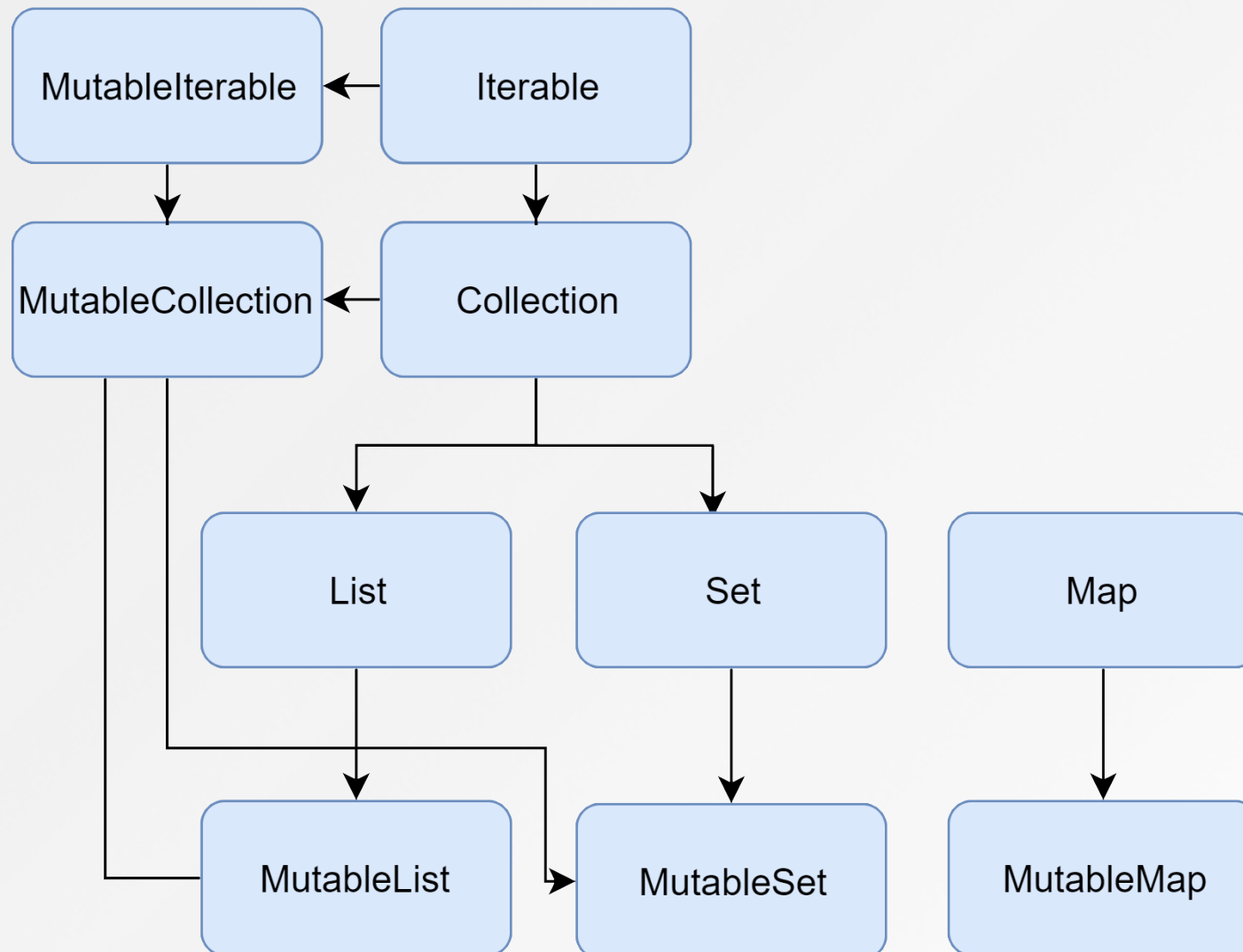
- » To throw an exception object, use the **throw**-expression
- » To catch an exception, use the **try**-expression

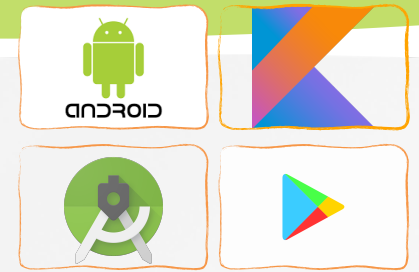
```
val sNumber = "12ba"  
  
try {  
    val number = sNumber as Int  
}  
catch (e:Exception) {  
    println(e.localizedMessage)  
}
```

```
// java.base/java.lang.String cannot be cast to java.base/  
java.lang.Integer
```



Collections



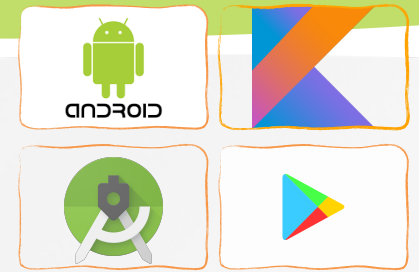


Get started with filters

» Filtering and Ordering

```
val list = mutableListOf(1, null, 2, null, 3, 4, 5, 6, 7, 8, 9)
    .filterNotNull()
    .filter { it % 2 == 0 }
    .sortedDescending()

println(list) //[8, 6, 4, 2]
```



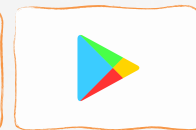
Get started with filters

» Filtering and Ordering

```
val decorations = listOf ("rock", "pagoda", "plastic plant",  
"alligator", "flowerpot")
```

```
fun main() {  
    println( decorations.filter {it[0] == 'p'})  
}
```

⇒ [pagoda, plastic plant]

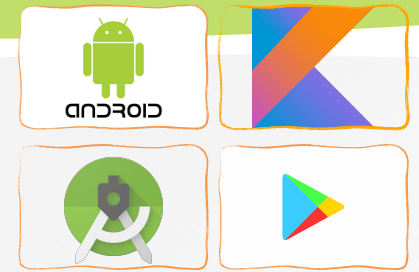


Collections (cont.)

Common operations

<code>getOrElse()</code>	<code>groupBy()</code>
<code>find()</code>	<code>map()</code>
<code>filter()</code>	<code>mapNotNull()</code>
<code>filterNot()</code>	<code>all()</code>
<code>filterNotNull()</code>	<code>any()</code>
<code>flatMap()</code>	<code>maxBy()</code>
<code>take()</code>	<code>minBy()</code>
<code>takeLast()</code>	<code>minWith()</code>
<code>sortBy()</code>	<code>sumBy()</code>
<code>sortByDescending()</code>	<code>zip()</code>

<https://kotlinlang.org/docs/reference/collection-operations.html>



Reference

- » <https://kotlinlang.org/docs/reference/>
- » <https://developer.android.com/courses/kotlin-bootcamp/overview>
- » Peter Späth, Learn Kotlin for Android Development, 2019

Assignments 1

Classes and Objects



» Which of the following appears to be a valid class declaration?

```
class Triangle(color:Int) (  
    val coordinates:Array<Pair<Double,Double>> = arrayOf()  
)  
  
class Triangle(color: Int) {  
    val coordinates: Array<Pair<Double, Double>> = arrayOf()  
}  
  
class simple_rectangle() {  
    val coordinates:Array<Pair<Double,Double>> = arrayOf()  
}  
  
class Colored Rectangle(color:Int) {  
    val coordinates:Array<Pair<Double,Double>> = arrayOf()  
}
```